

Development Activity 2 — Build a Role-Based Functional Prototype

Activity Objective

In the previous activity, you developed the **initial frontend prototype** of your semester project.

In this activity, you will take that prototype one step further by making it **role-aware and functionally interactive**.

You will use the user roles already identified in your project proposal and create a simple frontend experience in which different users see and access different functions.

Important: Continue working on the **same semester project**. Do not create a separate application.

This activity can be completed using your existing **HTML, CSS, and JavaScript files** without requiring internet access.

Part 1 — Select Two User Roles

Review the roles identified in your project.

Select **two different roles** that have clearly different responsibilities.

Example

For a Provincial Complaint Management System:

- Citizen
- Department Officer

For an Alumni Collaboration System:

- Alumni
- Administrator

Your Roles

Role 1: _____

Role 2: _____

Part 2 — Define What Each Role Can Do

Identify **at least 3 functions** for each role.

Role	Functions
Role 1	1. _____ 2. _____ 3. _____
Role 2	1. _____ 2. _____ 3. _____

Make sure the functions reflect the **actual responsibilities** of the selected roles.

Part 3 — Create Demo Users

Create simple demo users in your JavaScript code.

For example:

```
User 1:  
Name: Ali  
Role: Citizen
```

```
User 2:  
Name: Sara  
Role: Officer
```

You may use any names appropriate to your project.

You do **not** need a database for this activity.

Part 4 — Make the Login Page Functional

Use the login page developed in Activity 1.

Modify it so that a user can enter the credentials of one of your demo users.

After successful login:

The system should recognize the user's role and take the user to the appropriate interface.

For example:

```
Login
  ↓
Identify User
  ↓
Identify Role
  ↓
Display Appropriate Dashboard
```

A simple JavaScript implementation is sufficient.

Part 5 — Create Role-Specific Dashboards

Your system should now display different functions for the two selected roles.

Example

Citizen Dashboard

```
Welcome, Ali

[Submit Complaint]
[My Complaints]
[Track Complaint]
[Logout]
```

Officer Dashboard

```
Welcome, Sara

[Assigned Complaints]
[Update Complaint Status]
[View Department Reports]
[Logout]
```

The interface should reflect the **actual responsibilities of your project**.

Part 6 — Test the Difference Between the Roles

After implementing the two dashboards, test both users.

Complete the following table:

Test	Expected Result
Role 1 logs in	_____
Role 2 logs in	_____
Role 1 views its dashboard	_____
Role 2 views its dashboard	_____

The two roles should **not receive identical interfaces** when their responsibilities are different.

Part 7 — Protect a Function From the Wrong Role

Choose **one function** that should belong to only one of the selected roles.

For example:

Only an Officer can update complaint status.

or:

Only an Administrator can delete a user.

or:

Only an Alumni member can create a professional profile.

Modify your JavaScript so that an unauthorized role cannot perform that function.

Function Selected

Authorized Role

Example logic

```
IF user's role = authorized role
  perform function
ELSE
  display "Access Denied"
END IF
```

Use a simple implementation appropriate to your project.

Part 8 — Add One Security-Aware Data Display

Select **one piece of information that should not be unnecessarily exposed** to every user.

Examples:

- Personal phone number
- Email address
- CNIC
- Internal complaint notes
- Financial information
- Private messages
- Administrative information

Modify your interface so that the information is displayed only to the appropriate role, or displayed in a limited/masked form.

Selected Information

Who should be allowed to see it?

Part 9 — Add Logout

Add a **Logout** option to your interface.

When the user logs out:

- The current user/session information should be cleared.
 - The user should return to the login page.
 - The protected dashboard should no longer be displayed as the active interface.
-

Part 10 — Test Your Prototype

Perform the following tests:

Test 1 — Correct Role

Log in using a valid user.

Result: _____

Test 2 — Different Role

Log out and log in using the second user.

Result: _____

Test 3 — Restricted Function

Try to use the selected restricted function with the wrong role.

Result: _____

Test 4 — Logout

Log out from the application.

Result: _____

Part 11 — Security Development Note

Complete the following:

Security Concept Applied

Where did you apply it?

What did you change in your project?

What happens when an unauthorized role attempts the restricted operation?

Submission

Your project should contain:

- ✓ Same project selected in Activity 1
- ✓ Two functional user roles
- ✓ Functional login for demo users
- ✓ Role-specific dashboard/interface
- ✓ At least one role-restricted function
- ✓ One protected or limited piece of information
- ✓ Working logout
- ✓ Testing of both roles

Submit

Project Name: _____

Group Members: _____

Live/Local Project: _____

Roles Implemented: _____

Restricted Function: _____

Security Concept Applied: _____